Programowanie gier komputerowych 2025-2026

Wprowadzenie do programowania gier

inż. Mateusz Połeć

Semestr zimowy 2026/2027

Spis treści

1. Cel
2. Wykorzystane technologie
3. Konwencja kodu
4. Struktura projektu
5. Premake5
6. Catch2
7. ImGui
8. Spdlog
9. nlohmann/json
10. Skrypty
11. Repozytorium

Cel

Skupiamy się na skonfigurowaiu projektu – system budowania, zależności, inicjacja repozytorium w serwisie GitHub.

Wykorzystane technologie

- Do pracy nad grami preferujemy system operacyjny Windows.
- Pracujemy w standardzie C++20.
- System budowania dowolny, choć osobiście preferowałbym wybranie premake5 + Visual Studio w wersji 2022 lub 2026.
- Zależności i wykorzystane biblioteki to SFML w wersji 3.1.0, catch2, imgui, nlohmannjson oraz spdlog.

Konwencja kodu

Nie obowiązuje żadna konwencja, każdy programuje w takim "stylu", jaki mu odpowiada, natomiast dla czytelności i ułatwienia pracy z kodem nie tylko na tych zajęciach rekomenduje przyjęcie następujących zasad:

- Każda nazwa klasy poprzedzona jest literą ‘C‘, np. `CEntity` lub `CAssetsManager`.
- Każde pole w klasie poprzedzone jest konstrukcją ‘m_‘, np. `m_Health` lub `m_Points`.
- Każda nazwa enuma poprzedzona jest literką ‘E‘, np. `EPlayerModules` lub `EBufferType`.
- Każda nazwa struktury poprzedzona jest literką ‘S‘, np. `SAnimationFrame`.
- Każda klasa abstrakcyjna poprzedzona jest literką ‘I‘, np. `IEvent`.
- Każda klasa szablonowa poprzedzona jest literką ‘T‘, np. `TAsset`.
- Uwaga: Pola w strukturach nie powinny być poprzedzone prefiksem ‘m_‘!

Konwencja kodu

- Każda zmienna statystyczna powinna być poprzedzona prefiksem ‘s_‘ lub jej nazwa powinna zostać napisana w `UPPER_SNAKE_CASE`, np. `DEFAULT_FONT_SIZE`.
- Zawsze przechodzimy do następnej linii otwierając nowy zakres (ang. scope), np. `if (m_Initialized) { // Logic... }` jest niedozwolone!
- Aliasy na typ danych powinny kończyć się suffiksem ‘_t‘, np. `event_id_t`.
- Każdy argument funkcji powinien być poprzedzony prefiksem ‘_‘, np. `void Clamp01(int _val) {}`
- Zmienne powinny przyjmować styl `lower_snake_case`, np. `float distance_2_trigger`.
- Nazwa klasy, struktury i enuma powinny być pisane w `UpperCammelCase`, np. `CEventDispatcher`.

- Nazwy namespace'ów powinny być pisane w `lower_snake_case`, np. `namespace engine::time_managers`.
- Bezwzględnie stosujemy się do zasady `const-correctness`, jeżeli jakiś zasób nie jest modyfikowany, użyj `const`. Jeżeli jakiś zasób jest możliwy do ewaluacji w czasie kompilacji, użyj `constexpr`.
- Mile wiziane atrybuty np. `[[nodiscard]]` przy metodach zwracających wartości.
- Wszystkie `include` odwołujące się do bibliotek wewnętrznych STL i zewnętrznych w projekcie powinny być importowane jako pierwsze, używając ostrych nawiasów, np. `include <vector>`.
- Wszystkie `include` odwołujące się do kodu napisanego samodzielnie powinny być importowane jako drugie, używając cudzysłowia, np. `include "Game.h"`.
- Forward declarations są naturalne i ich nie unikniemy, jeżeli pojawia się problem z circular dependency, pomyśl o forward declaration.

Struktura projektu

- Folder **engine** - Tu będziemy przechowywać większość kodu (ECS, system Inputów etc.) i zewnętrzne zależności potrzebujące kompilacji (np. `.cpp` biblioteki `imgui`)
- Folder **tests** - Folder przeznaczony do wykonywania testów jednostkowych.
- Folder **assets** - Wszystkie zasoby nieprogramistyczne (zdjęcia kapibar, tekstury, czcionki, audio, konfiguracje inputów, presety do ECS).
- Folder **game** - Cały kod powiązany z grą. Głównie **main.cpp**, komponenty gry, systemy gracza.
- Folder **vendor** - Wszystkie zewnętrzne zależności, podzielony na trzy podfoldery:
  - **bin** - Zawierający wszystkie dynamiczne biblioteki (.dll)
  - **lib** - Zawierający wszystkie statyczne biblioteki (.lib)
  - **include** - Zawierający wszystkie pliki nagłówkowe zewnętrznych bibliotek.

Struktura projektu

- Skrypt **generate.ps1** oszczędzający czasu na wpisywanie komendy odpowiadającej za wygenerowanie solucji do Visual Studio.
- Skrypt **clean.ps1** czyszczący plik z metadanych wyprodukowanych przez Visual Studio i zależności powstałych przy kompilacji, jak i folderu **build**.
- Plik **premake5.lua** zawierający potrzebną konfigurację do poprawnego wygenerowania solucji dla Visual Studio.
- Executable **premake5.exe** potrzebny do wygenerowania solucji z wprowadzonej konfiguracji dostarczanej przez **premake5.lua**

Premake5

Premake5 to narzędzie do definiowania i generowania konfiguracji projektów C/C++. Zamiast ręcznie konfigurować projekty dla Visual Studio, Make, Xcode czy innych systemów budowania, opisujemy projekt w pliku premake5.lua, a Premake generuje odpowiednie pliki projektu. Link do strony z premake5, dostępna jest również dokumentacja: `https://premake.github.io/`

- workspace: Definiuje całe rozwiązanie / grupę projektów
- project: Definiuje konkretny projekt, np. aplikację lub bibliotekę
- configurations: Określa konfiguracje budowania, np. Debug, Release
- platforms: Określa docelowe warianty/platformy, np. x64 / MacOS.
- files: Wskazuje pliki źródłowe projektu

Premake5

- includedirs: - Dodaje ścieżki do nagłówków (.h, .hpp)
- libdirs: Dodaje ścieżki, w których znajdują się biblioteki
- links: Określa biblioteki, z którymi projekt ma być linkowany
- defines: Dodaje makra preprocesora, np. DEBUG
- filter: Pozwala stosować ustawienia tylko dla wybranej konfiguracji/platformy
- targetdir: Określa katalog, do którego trafia wynik kompilacji
- objdir: Określa katalog na pliki pośrednie (.obj, .o)

Premake5

```
workspace "MyProject"
    configurations { "Debug", "Release" }
    platforms { "x64" }

project "MyApp"
    kind "ConsoleApp"
    language "C++"

    files { "src/**.cpp", "include/**.h" }

    includedirs { "include" }

    defines { "MY_APP" }

    targetdir "bin/%{cfg.buildcfg}"
    objdir "bin-int/%{cfg.buildcfg}"

    filter "configurations:Debug"
        defines { "DEBUG" }
        symbols "On"

    filter "configurations:Release"
        optimize "On"
```

Catch2

Catch2 to framework do tworzenia i uruchamiania testów jednostkowych w C++.
Pozwala definiować testy bez konieczności tworzenia rozbudowanej infrastruktury testowej. Testy są zwykłym kodem C++, a Catch2 dostarcza mechanizmy do ich organizacji, uruchamiania oraz raportowania wyników.
Link do repozytorium: `https://github.com/catchorg/Catch2/`.

- TEST_CASE – Definiuje przypadek testowy
- SECTION Dzieli test na niezależne scenariusze
- REQUIRE Sprawdza warunek; błąd przerywa test
- CHECK Sprawdza warunek; test jest kontynuowany
- REQUIRE_FALSE Sprawdza, że warunek jest fałszywy
- CHECK_FALSE Jak CHECK, ale dla warunku fałszywego
- REQUIRE_THROWS Sprawdza, czy kod rzuca wyjątek
- REQUIRE_NOTHROW – Sprawdza, czy kod nie rzuca wyjątku
- GENERATE – Dostarcza wiele wartości wejściowych do testu

Catch2 — prosty test

```
#include <catch2/catch2.hpp>

int add(int a, int b)
{
    return a + b;
}

TEST_CASE("Addition works")
{
    REQUIRE(add(2, 3) == 5);
    REQUIRE(add(-1, 1) == 0);
}
```

ImGui

Dear ImGui to lekka biblioteka GUI dla C++, przeznaczona przede wszystkim do tworzenia narzędzi, edytorów, debug UI oraz interfejsów aplikacji czasu rzeczywistego.
W przeciwieństwie do klasycznych bibliotek GUI, ImGui wykorzystuje model Immediate Mode — interfejs opisujemy w kodzie podczas każdej klatki.
Link do repozytorium: `https://github.com/ocornut/imgui`

- ImGui::Begin() – Rozpoczyna okno
- ImGui::End() – Kończy okno
- ImGui::Text() – Wyświetla tekst
- ImGui::Button() – Tworzy przycisk
- ImGui::Checkbox() – Tworzy pole wyboru
- ImGui::SliderFloat() – Tworzy suwak
- ImGui::InputText() – Pole tekstowe
- ImGui::SameLine() – Umieszcza element w tej samej linii
- ImGui::BeginChild() – Tworzy zagnieżdżony obszar

ImGui

```
static bool enabled = false;
static float volume = 0.75f;

ImGui::Begin("Settings");

ImGui::Checkbox("Enabled", &enabled);
ImGui::SliderFloat(
    "Volume",
    &volume,
    0.0f,
    1.0f
);

if (enabled)
    ImGui::Text("System is enabled");

ImGui::End();
```

Spdlog

spdlog to szybka i lekka biblioteka do logowania w C++.
Pozwala w prosty sposób zapisywać informacje o działaniu programu — zarówno na konsolę, jak i do plików. Obsługuje m.in. różne poziomy logowania, formatowanie wiadomości, loggery oraz logowanie asynchroniczne.
Link do repozytorium: `https://github.com/gabime/spdlog`

- spdlog::info() – Informacja o działaniu programu
- spdlog::debug() – Informacje pomocne podczas debugowania
- spdlog::warn() – Ostrzeżenie
- spdlog::error() Błąd
- spdlog::critical() – Poważny błąd
- spdlog::trace() – Bardzo szczegółowe informacje
- spdlog::set_level() – Ustawia minimalny poziom logowania
- spdlog::set_pattern() Ustawia format wiadomości
- spdlog::basic_logger_mt() Tworzy logger zapisujący do pliku
- spdlog::stdout_color_mt() – Tworzy logger dla kolorowej konsoli

Spdlog

```
#include <spdlog/spdlog.h>

int main()
{
    spdlog::set_pattern(
        "[%H:%M:%S] [%^%l%$] %v"
    );

    spdlog::info("Server started");
    spdlog::warn("Connection is slow");
    spdlog::error("Connection lost");

    return 0;
}
```

Przykładowy wynik:

```
[14:32:05] [info] Server started
[14:32:07] [warning] Connection is slow
[14:32:10] [error] Connection lost
```

nlohmann/json

nlohmann/json to popularna biblioteka C++ umożliwiająca pracę z formatem JSON. Pozwala tworzyć obiekty JSON, odczytywać dane, modyfikować je oraz konwertować pomiędzy JSON-em a obiektami C++. Link do repozytorium: `https://github.com/nlohmann/json`

- json – Podstawowy typ reprezentujący dane JSON
- operator[] – Dostęp do elementów obiektu lub tablicy
- at() – Dostęp do elementu z kontrolą istnienia
- value() – Pobranie wartości z wartością domyślną
- contains() – Sprawdzenie, czy istnieje określony klucz
- push_back() – Dodanie elementu do tablicy
- dump() – Konwersja JSON $\rightarrow$ string
- parse() – Konwersja string $\rightarrow$ JSON
- get<T>() – Konwersja JSON $\rightarrow$ typ C++
- get_to() – Wczytanie wartości JSON do zmiennej C++

nlohmann/json

```
#include <nlohmann/json.hpp>

using json = nlohmann::json;

int main()
{
    json player;

    player["name"] = "John";
    player["level"] = 10;
    player["health"] = 100.0;

    std::cout << player["name"] << '\n';
    std::cout << player["level"] << '\n';
}
```

Skrypty

Proponuję przygotować sobie dwa pomocnicze skrypty:

- pierwszy ułatwiający generowanie solucji dla Visual Studio,
- drugi czyszczący repozytorium z metadanych oraz plików związanych z kompilacją, w tym folderu **build/**.

Zawartość skryptu `generate.ps1`

```
./premake5 vs2022 -file=./premake5.lua
```

Natomiast skrypt czyszczący zostawiam do samodzielnej pracy, zostawiając tylko przykład, jak zostało to zrobione w przykładowym rozwiązaniu listy zadań. `https://github.com/mateuszpolec/uwr-pgier/blob/main/2026/l1/clean.ps1`

Repozytorium

Inicjalizacja repozytorium

```
git init
```

Dodanie plik w

```
git add .
```

Pierwszy commit

```
git commit -m "Initial commit"
```

Dodanie zdalnego repozytorium

```
git remote add origin <URL>
```

Wysylanie zmian

```
git branch -M main
git push -u origin main
```

Repozytorium

Nie zapomijcie o .gitignore! `https://github.com/mateuszpolec/uwr-pgier/blob/main/.gitignore`